

CHAPTER 15

Hashing and Collision

LEARNING OBJECTIVE

In this chapter, we will discuss another data structure known as hash table. We will see what a hash table is and why do we prefer hash tables over simple arrays. We will also discuss hash functions, collisions, and the techniques to resolve collisions.

15.1 INTRODUCTION

In Chapter 14, we discussed two search algorithms: *linear search* and *binary search*. Linear search has a running time proportional to $O(n)$, while binary search takes time proportional to $O(\log n)$, where n is the number of elements in the array. Binary search and binary search trees are efficient algorithms to search for an element. But what if we want to perform the search operation in time proportional to $O(1)$? In other words, is there a way to search an array in constant time, irrespective of its size?

Key	Array of Employees' Records
Key 0 → [0]	Employee record with Emp_ID 0
Key 1 → [1]	Employee record with Emp_ID 1
Key 2 → [2]	Employee record with Emp_ID 2
.....	
.....	
Key 98 → [98]	Employee record with Emp_ID 98
Key 99 → [99]	Employee record with Emp_ID 99

Figure 15.1 Records of employees

There are two solutions to this problem. Let us take an example to explain the first solution. In a small company of 100 employees, each employee is assigned an `Emp_ID` in the range 0–99. To store the records in an array, each employee's `Emp_ID` acts as an index into the array where the employee's record will be stored as shown in Fig. 15.1.

In this case, we can directly access the record of any employee, once we know his `Emp_ID`, because the array index is the same as the `Emp_ID` number. But practically, this implementation is hardly feasible.

Let us assume that the same company uses a five-digit `Emp_ID` as the primary key. In this case, key values will range from 00000 to 99999. If we want to use the same technique as above, we need an array of size 100,000, of which only 100 elements will be used. This is illustrated in Fig. 15.2.

Key	Array of Employees' Records
Key 00000 → [0]	Employee record with Emp_ID 00000
.....	
Key n → [n]	Employee record with Emp_ID n
.....	
Key 99998 → [99998]	Employee record with Emp_ID 99998
Key 99999 → [99999]	Employee record with Emp_ID 99999

Figure 15.2 Records of employees with a five-digit `Emp_ID`

It is impractical to waste so much storage space just to ensure that each employee's record is in a unique and predictable location.

Whether we use a two-digit primary key (`Emp_ID`) or a five-digit key, there are just 100 employees in the company. Thus, we will be using only 100 locations in the array. Therefore, in order to keep the array size down to the size that we will actually be using (100 elements), another good option is to use just the last two digits of the key to identify each employee. For example, the employee with `Emp_ID` 79439 will be stored in the element of the array with index 39. Similarly, the employee with `Emp_ID` 12345 will have his record stored in the array at the 45th location.

In the second solution, the elements are not stored according to the *value* of the key. So in this case, we need a way to convert a five-digit key number to a two-digit array index. We need a function which will do the transformation. In this case, we will use the term *hash table* for an array and the function that will carry out the transformation will be called a *hash function*.

15.2 HASH TABLES

Hash table is a data structure in which keys are mapped to array positions by a hash function. In the example discussed here we will use a hash function that extracts the last two digits of the key. Therefore, we map the keys to array locations or array indices. A value stored in a hash table can be searched in $O(1)$ time by using a hash function which generates an address from the key (by producing the index of the array where the value is stored).

Figure 15.3 shows a direct correspondence between the keys and the indices of the array. This concept is useful when the total universe of keys is small and when most of the keys are actually used from the whole set of keys. This is equivalent to our first example, where there are 100 keys for 100 employees.

However, when the set κ of keys that are actually used is smaller than the universe of keys ($\mathcal{U}$), a hash table consumes less storage space. The storage requirement for a hash table is $O(\kappa)$, where κ is the number of keys actually used.

In a hash table, an element with key κ is stored at index $h(\kappa)$ and not κ . It means a hash function h is used to calculate the index at which the element with key κ will be stored. This process of mapping the keys to appropriate locations (or indices) in a hash table is called *hashing*.

Figure 15.4 shows a hash table in which each key from the set κ is mapped to locations generated by using a hash function. Note that keys κ_2 and κ_6 point to the same memory location. This is known as *collision*. That is, when two or more keys map to the same memory location, a collision

is said to occur. Similarly, keys k_5 and k_7 also collide. The main goal of using a hash function is to reduce the range of array indices that have to be handled. Thus, instead of having u values, we just need k values, thereby reducing the amount of storage space required.

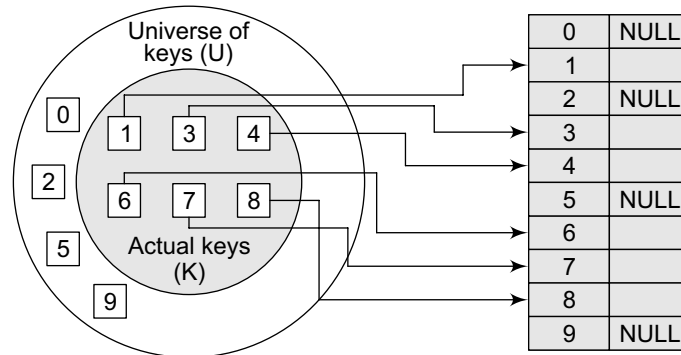


Figure 15.3 Direct relationship between key and index in the array

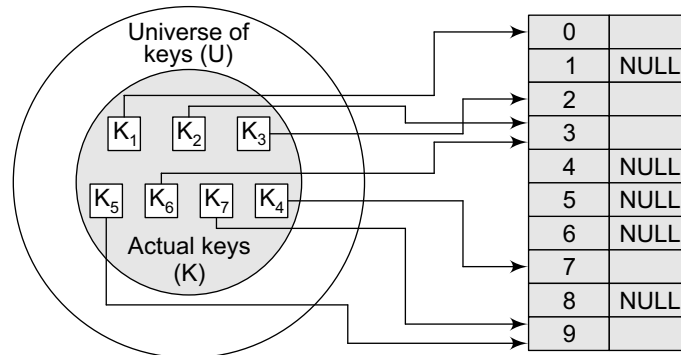


Figure 15.4 Relationship between keys and hash table index

15.3 HASH FUNCTIONS

A hash function is a mathematical formula which, when applied to a key, produces an integer which can be used as an index for the key in the hash table. The main aim of a hash function is that elements should be relatively, randomly, and uniformly distributed. It produces a unique set of integers within some suitable range in order to reduce the number of collisions. In practice, there is no hash function that eliminates collisions completely. A good hash function can only minimize the number of collisions by spreading the elements uniformly throughout the array.

In this section, we will discuss the popular hash functions which help to minimize collisions. But before that, let us first look at the properties of a good hash function.

Properties of a Good Hash Function

Low cost The cost of executing a hash function must be small, so that using the hashing technique becomes preferable over other approaches. For example, if binary search algorithm can search an element from a sorted table of n items with $\log_2 n$ key comparisons, then the hash function must cost less than performing $\log_2 n$ key comparisons.

Determinism A hash procedure must be deterministic. This means that the same hash value must be generated for a given input value. However, this criteria excludes hash functions that depend

on external variable parameters (such as the time of day) and on the memory address of the object being hashed (because address of the object may change during processing).

Uniformity A good hash function must map the keys as evenly as possible over its output range. This means that the probability of generating every hash value in the output range should roughly be the same. The property of uniformity also minimizes the number of collisions.

15.4 DIFFERENT HASH FUNCTIONS

In this section, we will discuss the hash functions which use numeric keys. However, there can be cases in real-world applications where we can have alphanumeric keys rather than simple numeric keys. In such cases, the ASCII value of the character can be used to transform it into its equivalent numeric key. Once this transformation is done, any of the hash functions given below can be applied to generate the hash value.

15.4.1 Division Method

It is the most simple method of hashing an integer x . This method divides x by M and then uses the remainder obtained. In this case, the hash function can be given as

$$h(x) = x \bmod M$$

The division method is quite good for just about any value of M and since it requires only a single division operation, the method works very fast. However, extra care should be taken to select a suitable value for M .

For example, suppose M is an even number then $h(x)$ is even if x is even and $h(x)$ is odd if x is odd. If all possible keys are equi-probable, then this is not a problem. But if even keys are more likely than odd keys, then the division method will not spread the hashed values uniformly.

Generally, it is best to choose M to be a prime number because making M a prime number increases the likelihood that the keys are mapped with a uniformity in the output range of values. M should also be not too close to the exact powers of 2. If we have

$$h(x) = x \bmod 2^k$$

then the function will simply extract the lowest k bits of the binary representation of x .

The division method is extremely simple to implement. The following code segment illustrates how to do this:

```
int const M = 97; // a prime number
int h (int x)
{ return (x % M); }
```

A potential drawback of the division method is that while using this method, consecutive keys map to consecutive hash values. On one hand, this is good as it ensures that consecutive keys do not collide, but on the other, it also means that consecutive array locations will be occupied. This may lead to degradation in performance.

Example 15.1 Calculate the hash values of keys 1234 and 5462.

Solution Setting $M = 97$, hash values can be calculated as:

$$\begin{aligned} h(1234) &= 1234 \% 97 = 70 \\ h(5462) &= 5462 \% 97 = 16 \end{aligned}$$

15.4.2 Multiplication Method

The steps involved in the multiplication method are as follows:

Step 1: Choose a constant A such that $0 < A < 1$.

Step 2: Multiply the key k by A .

Step 3: Extract the fractional part of kA .

Step 4: Multiply the result of Step 3 by the size of hash table (m).

Hence, the hash function can be given as:

$$h(k) = \lfloor m (kA \bmod 1) \rfloor$$

where $(kA \bmod 1)$ gives the fractional part of kA and m is the total number of indices in the hash table.

The greatest advantage of this method is that it works practically with any value of A . Although the algorithm works better with some values, the optimal choice depends on the characteristics of the data being hashed. Knuth has suggested that the best choice of A is

$$(\sqrt{5} - 1) / 2 = 0.6180339887$$

Example 15.2 Given a hash table of size 1000, map the key 12345 to an appropriate location in the hash table.

Solution We will use $A = 0.618033$, $m = 1000$, and $k = 12345$

$$\begin{aligned} h(12345) &= \lfloor 1000 (12345 \times 0.618033 \bmod 1) \rfloor \\ &= \lfloor 1000 (7629.617385 \bmod 1) \rfloor \\ &= \lfloor 1000 (0.617385) \rfloor \\ &= \lfloor 617.385 \rfloor \\ &= 617 \end{aligned}$$

15.4.3 Mid-Square Method

The mid-square method is a good hash function which works in two steps:

Step 1: Square the value of the key. That is, find k^2 .

Step 2: Extract the middle r digits of the result obtained in Step 1.

The algorithm works well because most or all digits of the key value contribute to the result. This is because all the digits in the original key value contribute to produce the middle digits of the squared value. Therefore, the result is not dominated by the distribution of the bottom digit or the top digit of the original key value.

In the mid-square method, the same r digits must be chosen from all the keys. Therefore, the hash function can be given as:

$$h(k) = s$$

where s is obtained by selecting r digits from k^2 .

Example 15.3 Calculate the hash value for keys 1234 and 5642 using the mid-square method. The hash table has 100 memory locations.

Solution Note that the hash table has 100 memory locations whose indices vary from 0 to 99. This means that only two digits are needed to map the key to a location in the hash table, so $r = 2$.

When $k = 1234$, $k^2 = 1522756$, $h(1234) = 27$

When $k = 5642$, $k^2 = 31832164$, $h(5642) = 21$

Observe that the 3rd and 4th digits starting from the right are chosen.

15.4.4 Folding Method

The folding method works in the following two steps:

Step 1: Divide the key value into a number of parts. That is, divide k into parts $k_1, k_2, \dots, k_n$, where each part has the same number of digits except the last part which may have lesser digits than the other parts.

Step 2: Add the individual parts. That is, obtain the sum of $k_1 + k_2 + \dots + k_n$. The hash value is produced by ignoring the last carry, if any.

Note that the number of digits in each part of the key will vary depending upon the size of the hash table. For example, if the hash table has a size of 1000, then there are 1000 locations in the hash table. To address these 1000 locations, we need at least three digits; therefore, each part of the key must have three digits except the last part which may have lesser digits.

Example 15.4 Given a hash table of 100 locations, calculate the hash value using folding method for keys 5678, 321, and 34567.

Solution

Since there are 100 memory locations to address, we will break the key into parts where each part (except the last) will contain two digits. The hash values can be obtained as shown below:

key	5678	321	34567
Parts	56 and 78	32 and 1	34, 56 and 7
Sum	134	33	97
Hash value	34 (ignore the last carry)	33	97

15.5 COLLISIONS

As discussed earlier in this chapter, collisions occur when the hash function maps two different keys to the same location. Obviously, two records cannot be stored in the same location. Therefore, a method used to solve the problem of collision, also called *collision resolution technique*, is applied. The two most popular methods of resolving collisions are:

1. Open addressing
2. Chaining

In this section, we will discuss both these techniques in detail.

15.5.1 Collision Resolution by Open Addressing

Once a collision takes place, open addressing or closed hashing computes new positions using a probe sequence and the next record is stored in that position. In this technique, all the values are stored in the hash table. The hash table contains two types of values: *sentinel values* (e.g., -1) and *data values*. The presence of a sentinel value indicates that the location contains no data value at present but can be used to hold a value.

When a key is mapped to a particular memory location, then the value it holds is checked. If it contains a sentinel value, then the location is free and the data value can be stored in it. However, if the location already has some data value stored in it, then other slots are examined systematically in the forward direction to find a free slot. If even a single free location is not found, then we have an **OVERFLOW** condition.

The process of examining memory locations in the hash table is called *probing*. Open addressing technique can be implemented using linear probing, quadratic probing, double hashing, and rehashing.

Linear Probing

The simplest approach to resolve a collision is linear probing. In this technique, if a value is already stored at a location generated by $h(k)$, then the following hash function is used to resolve the collision:

$$h(k, i) = [h'(k) + i] \bmod m$$

Where m is the size of the hash table, $h'(k) = (k \bmod m)$, and i is the probe number that varies from 0 to $m-1$.

Therefore, for a given key k , first the location generated by $[h'(k) \bmod m]$ is probed because for the first time $i=0$. If the location is free, the value is stored in it, else the second probe generates the address of the location given by $[h'(k) + 1] \bmod m$. Similarly, if the location is occupied, then subsequent probes generate the address as $[h'(k) + 2] \bmod m$, $[h'(k) + 3] \bmod m$, $[h'(k) + 4] \bmod m$, $[h'(k) + 5] \bmod m$, and so on, until a free location is found.

Note Linear probing is known for its simplicity. When we have to store a value, we try the slots: $[h'(k)] \bmod m$, $[h'(k) + 1] \bmod m$, $[h'(k) + 2] \bmod m$, $[h'(k) + 3] \bmod m$, $[h'(k) + 4] \bmod m$, $[h'(k) + 5] \bmod m$, and so on, until a vacant location is found.

Example 15.5 Consider a hash table of size 10. Using linear probing, insert the keys 72, 27, 36, 24, 63, 81, 92, and 101 into the table.

Let $h'(k) = k \bmod m$, $m = 10$

Initially, the hash table can be given as:

0	1	2	3	4	5	6	7	8	9
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1

Step 1 Key = 72

$$\begin{aligned} h(72, 0) &= (72 \bmod 10 + 0) \bmod 10 \\ &= (2) \bmod 10 \\ &= 2 \end{aligned}$$

Since $\tau[2]$ is vacant, insert key 72 at this location.

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	-1	-1	-1

Step 2 Key = 27

$$\begin{aligned} h(27, 0) &= (27 \bmod 10 + 0) \bmod 10 \\ &= (7) \bmod 10 \\ &= 7 \end{aligned}$$

Since $\tau[7]$ is vacant, insert key 27 at this location.

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	27	-1	-1

Step 3 Key = 36

$$\begin{aligned} h(36, 0) &= (36 \bmod 10 + 0) \bmod 10 \\ &= (6) \bmod 10 \\ &= 6 \end{aligned}$$

Since $\tau[6]$ is vacant, insert key 36 at this location.

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	36	27	-1	-1

Step 4 Key = 24

$$h(24, 0) = (24 \bmod 10 + 0) \bmod 10$$

$$= (4) \bmod 10$$

$$= 4$$

Since $\tau[4]$ is vacant, insert key 24 at this location.

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	24	-1	36	27	-1	-1

Step 5

Key = 63

$$h(63, 0) = (63 \bmod 10 + 0) \bmod 10$$

$$= (3) \bmod 10$$

$$= 3$$

Since $\tau[3]$ is vacant, insert key 63 at this location.

0	1	2	3	4	5	6	7	8	9
-1	-1	72	63	24	-1	36	27	-1	-1

Step 6

Key = 81

$$h(81, 0) = (81 \bmod 10 + 0) \bmod 10$$

$$= (1) \bmod 10$$

$$= 1$$

Since $\tau[1]$ is vacant, insert key 81 at this location.

0	1	2	3	4	5	6	7	8	9
0	81	72	63	24	-1	36	27	-1	-1

Step 7

Key = 92

$$h(92, 0) = (92 \bmod 10 + 0) \bmod 10$$

$$= (2) \bmod 10$$

$$= 2$$

Now $\tau[2]$ is occupied, so we cannot store the key 92 in $\tau[2]$. Therefore, try again for the next location. Thus probe, $i = 1$, this time.

Key = 92

$$h(92, 1) = (92 \bmod 10 + 1) \bmod 10$$

$$= (2 + 1) \bmod 10$$

$$= 3$$

Now $\tau[3]$ is occupied, so we cannot store the key 92 in $\tau[3]$. Therefore, try again for the next location. Thus probe, $i = 2$, this time.

Key = 92

$$h(92, 2) = (92 \bmod 10 + 2) \bmod 10$$

$$= (2 + 2) \bmod 10$$

$$= 4$$

Now $\tau[4]$ is occupied, so we cannot store the key 92 in $\tau[4]$. Therefore, try again for the next location. Thus probe, $i = 3$, this time.

Key = 92

$$h(92, 3) = (92 \bmod 10 + 3) \bmod 10$$

$$= (2 + 3) \bmod 10$$

$$= 5$$

Since $\tau[5]$ is vacant, insert key 92 at this location.

0	1	2	3	4	5	6	7	8	9
-1	81	72	63	24	92	36	27	-1	-1

Step 8

Key = 101

$$\begin{aligned}
 h(101, 0) &= (101 \bmod 10 + 0) \bmod 10 \\
 &= (1) \bmod 10 \\
 &= 1
 \end{aligned}$$

Now $\tau[1]$ is occupied, so we cannot store the key 101 in $\tau[1]$. Therefore, try again for the next location. Thus probe, $i = 1$, this time.

Key = 101

$$\begin{aligned}
 h(101, 1) &= (101 \bmod 10 + 1) \bmod 10 \\
 &= (1 + 1) \bmod 10 \\
 &= 2
 \end{aligned}$$

$\tau[2]$ is also occupied, so we cannot store the key in this location. The procedure will be repeated until the hash function generates the address of location 8 which is vacant and can be used to store the value in it.

Searching a Value using Linear Probing

The procedure for searching a value in a hash table is same as for storing a value in a hash table.

While searching for a value in a hash table, the array index is re-computed and the key of the element stored at that location is compared with the value that has to be searched. If a match is found, then the search operation is successful. The search time in this case is given as $O(1)$. If the key does not match, then the search function begins a sequential search of the array that continues until:

- the value is found, or
- the search function encounters a vacant location in the array, indicating that the value is not present, or
- the search function terminates because it reaches the end of the table and the value is not present.

In the worst case, the search operation may have to make $n-1$ comparisons, and the running time of the search algorithm may take $O(n)$ time. The worst case will be encountered when after scanning all the $n-1$ elements, the value is either present at the last location or not present in the table.

Thus, we see that with the increase in the number of collisions, the distance between the array index computed by the hash function and the actual location of the element increases, thereby increasing the search time.

Pros and Cons

Linear probing finds an empty location by doing a linear search in the array beginning from position $h(k)$. Although the algorithm provides good memory caching through good locality of reference, the drawback of this algorithm is that it results in clustering, and thus there is a higher risk of more collisions where one collision has already taken place. The performance of linear probing is sensitive to the distribution of input values.

As the hash table fills, clusters of consecutive cells are formed and the time required for a search increases with the size of the cluster. In addition to this, when a new value has to be inserted into the table at a position which is already occupied, that value is inserted at the end of the cluster, which again increases the length of the cluster. Generally, an insertion is made between two clusters that are separated by one vacant location. But with linear probing, there are more chances that subsequent insertions will also end up in one of the clusters, thereby potentially increasing the cluster length by an amount much greater than one. More the number of collisions, higher the

probes that are required to find a free location and lesser is the performance. This phenomenon is called *primary clustering*. To avoid primary clustering, other techniques such as quadratic probing and double hashing are used.

Quadratic Probing

In this technique, if a value is already stored at a location generated by $h(k)$, then the following hash function is used to resolve the collision:

$$h(k, i) = [h'(k) + c_1 i + c_2 i^2] \bmod m$$

where m is the size of the hash table, $h'(k) = (k \bmod m)$, i is the probe number that varies from 0 to $m-1$, and c_1 and c_2 are constants such that c_1 and $c_2 \neq 0$.

Quadratic probing eliminates the primary clustering phenomenon of linear probing because instead of doing a linear search, it does a quadratic search. For a given key k , first the location generated by $h'(k) \bmod m$ is probed. If the location is free, the value is stored in it, else subsequent locations probed are offset by factors that depend in a quadratic manner on the probe number i . Although quadratic probing performs better than linear probing, in order to maximize the utilization of the hash table, the values of c_1 , c_2 , and m need to be constrained.

Example 15.6 Consider a hash table of size 10. Using quadratic probing, insert the keys 72, 27, 36, 24, 63, 81, and 101 into the table. Take $c_1 = 1$ and $c_2 = 3$.

Solution

Let $h'(k) = k \bmod m$, $m = 10$

Initially, the hash table can be given as:

0	1	2	3	4	5	6	7	8	9
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1

We have,

$$h(k, i) = [h'(k) + c_1 i + c_2 i^2] \bmod m$$

Step 1 Key = 72

$$\begin{aligned} h(72, 0) &= [72 \bmod 10 + 1 \times 0 + 3 \times 0] \bmod 10 \\ &= [72 \bmod 10] \bmod 10 \\ &= 2 \bmod 10 \\ &= 2 \end{aligned}$$

Since $\tau[2]$ is vacant, insert the key 72 in $\tau[2]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	-1	-1	-1

Step 2 Key = 27

$$\begin{aligned} h(27, 0) &= [27 \bmod 10 + 1 \times 0 + 3 \times 0] \bmod 10 \\ &= [27 \bmod 10] \bmod 10 \\ &= 7 \bmod 10 \\ &= 7 \end{aligned}$$

Since $\tau[7]$ is vacant, insert the key 27 in $\tau[7]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	27	-1	-1

Step 3 Key = 36

$$\begin{aligned}
 h(36, 0) &= [36 \bmod 10 + 1 \times 0 + 3 \times 0] \bmod 10 \\
 &= [36 \bmod 10] \bmod 10 \\
 &= 6 \bmod 10 \\
 &= 6
 \end{aligned}$$

Since $\tau[6]$ is vacant, insert the key 36 in $\tau[6]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	36	27	-1	-1

Step 4 Key = 24

$$\begin{aligned}
 h(24, 0) &= [24 \bmod 10 + 1 \times 0 + 3 \times 0] \bmod 10 \\
 &= [24 \bmod 10] \bmod 10 \\
 &= 4 \bmod 10 \\
 &= 4
 \end{aligned}$$

Since $\tau[4]$ is vacant, insert the key 24 in $\tau[4]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	24	-1	36	27	-1	-1

Step 5 Key = 63

$$\begin{aligned}
 h(63, 0) &= [63 \bmod 10 + 1 \times 0 + 3 \times 0] \bmod 10 \\
 &= [63 \bmod 10] \bmod 10 \\
 &= 3 \bmod 10 \\
 &= 3
 \end{aligned}$$

Since $\tau[3]$ is vacant, insert the key 63 in $\tau[3]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	63	24	-1	36	27	-1	-1

Step 6 Key = 81

$$\begin{aligned}
 h(81, 0) &= [81 \bmod 10 + 1 \times 0 + 3 \times 0] \bmod 10 \\
 &= [81 \bmod 10] \bmod 10 \\
 &= 1 \bmod 10 \\
 &= 1
 \end{aligned}$$

Since $\tau[1]$ is vacant, insert the key 81 in $\tau[1]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	81	72	63	24	-1	36	27	-1	-1

Step 7 Key = 101

$$\begin{aligned}
 h(101, 0) &= [101 \bmod 10 + 1 \times 0 + 3 \times 0] \bmod 10 \\
 &= [101 \bmod 10 + 0] \bmod 10 \\
 &= 1 \bmod 10 \\
 &= 1
 \end{aligned}$$

Since $\tau[1]$ is already occupied, the key 101 cannot be stored in $\tau[1]$. Therefore, try again for next location. Thus probe, $i = 1$, this time.

Key = 101

$$h(101, 1) = [101 \bmod 10 + 1 \times 1 + 3 \times 1] \bmod 10$$

$$\begin{aligned}
 &= [101 \bmod 10 + 1 + 3] \bmod 10 \\
 &= [101 \bmod 10 + 4] \bmod 10 \\
 &= [1 + 4] \bmod 10 \\
 &= 5 \bmod 10 \\
 &= 5
 \end{aligned}$$

Since $\tau[5]$ is vacant, insert the key 101 in $\tau[5]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	81	72	63	24	101	36	27	-1	-1

Searching a Value using Quadratic Probing

While searching a value using the quadratic probing technique, the array index is re-computed and the key of the element stored at that location is compared with the value that has to be searched. If the desired key value matches with the key value at that location, then the element is present in the hash table and the search is said to be successful. In this case, the search time is given as $O(1)$. However, if the value does not match, then the search function begins a sequential search of the array that continues until:

- the value is found, or
- the search function encounters a vacant location in the array, indicating that the value is not present, or
- the search function terminates because it reaches the end of the table and the value is not present.

In the worst case, the search operation may take $n-1$ comparisons, and the running time of the search algorithm may be $O(n)$. The worst case will be encountered when after scanning all the $n-1$ elements, the value is either present at the last location or not present in the table.

Thus, we see that with the increase in the number of collisions, the distance between the array index computed by the hash function and the actual location of the element increases, thereby increasing the search time.

Pros and Cons

Quadratic probing resolves the primary clustering problem that exists in the linear probing technique. Quadratic probing provides good memory caching because it preserves some locality of reference. But linear probing does this task better and gives a better cache performance.

One of the major drawbacks of quadratic probing is that a sequence of successive probes may only explore a fraction of the table, and this fraction may be quite small. If this happens, then we will not be able to find an empty location in the table despite the fact that the table is by no means full. In Example 15.6 try to insert the key 92 and you will encounter this problem.

Although quadratic probing is free from primary clustering, it is still liable to what is known as *secondary clustering*. It means that if there is a collision between two keys, then the same probe sequence will be followed for both. With quadratic probing, the probability for multiple collisions increases as the table becomes full. This situation is usually encountered when the hash table is more than full.

Quadratic probing is widely applied in the Berkeley Fast File System to allocate free blocks.

Double Hashing

To start with, double hashing uses one hash value and then repeatedly steps forward an interval until an empty location is reached. The interval is decided using a second, independent hash function,

hence the name *double hashing*. In double hashing, we use two hash functions rather than a single function. The hash function in the case of double hashing can be given as:

$$h(k, i) = [h_1(k) + ih_2(k)] \bmod m$$

where m is the size of the hash table, $h_1(k)$ and $h_2(k)$ are two hash functions given as $h_1(k) = k \bmod m$, $h_2(k) = k \bmod m'$, i is the probe number that varies from 0 to $m-1$, and m' is chosen to be less than m . We can choose $m' = m-1$ or $m-2$.

When we have to insert a key k in the hash table, we first probe the location given by applying $[h_1(k) \bmod m]$ because during the first probe, $i = 0$. If the location is vacant, the key is inserted into it, else subsequent probes generate locations that are at an offset of $[h_2(k) \bmod m]$ from the previous location. Since the offset may vary with every probe depending on the value generated by the second hash function, the performance of double hashing is very close to the performance of the ideal scheme of uniform hashing.

Pros and Cons

Double hashing minimizes repeated collisions and the effects of clustering. That is, double hashing is free from problems associated with primary clustering as well as secondary clustering.

Example 15.7 Consider a hash table of size = 10. Using double hashing, insert the keys 72, 27, 36, 24, 63, 81, 92, and 101 into the table. Take $h_1 = (k \bmod 10)$ and $h_2 = (k \bmod 8)$.

Solution

Let $m = 10$

Initially, the hash table can be given as:

0	1	2	3	4	5	6	7	8	9
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1

We have,

$$h(k, i) = [h_1(k) + ih_2(k)] \bmod m$$

Step 1

Key = 72

$$\begin{aligned} h(72, 0) &= [72 \bmod 10 + (0 \times 72 \bmod 8)] \bmod 10 \\ &= [2 + (0 \times 0)] \bmod 10 \\ &= 2 \bmod 10 \\ &= 2 \end{aligned}$$

Since $\tau[2]$ is vacant, insert the key 72 in $\tau[2]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	-1	-1	-1

Step 2

Key = 27

$$\begin{aligned} h(27, 0) &= [27 \bmod 10 + (0 \times 27 \bmod 8)] \bmod 10 \\ &= [7 + (0 \times 3)] \bmod 10 \\ &= 7 \bmod 10 \\ &= 7 \end{aligned}$$

Since $\tau[7]$ is vacant, insert the key 27 in $\tau[7]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	27	-1	-1

Step 3 Key = 36

$$\begin{aligned}
 h(36, 0) &= [36 \bmod 10 + (0 \times 36 \bmod 8)] \bmod 10 \\
 &= [6 + (0 \times 4)] \bmod 10 \\
 &= 6 \bmod 10 \\
 &= 6
 \end{aligned}$$

Since $\tau[6]$ is vacant, insert the key 36 in $\tau[6]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	36	27	-1	-1

Step 4 Key = 24

$$\begin{aligned}
 h(24, 0) &= [24 \bmod 10 + (0 \times 24 \bmod 8)] \bmod 10 \\
 &= [4 + (0 \times 0)] \bmod 10 \\
 &= 4 \bmod 10 \\
 &= 4
 \end{aligned}$$

Since $\tau[4]$ is vacant, insert the key 24 in $\tau[4]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	24	-1	36	27	-1	-1

Step 5 Key = 63

$$\begin{aligned}
 h(63, 0) &= [63 \bmod 10 + (0 \times 63 \bmod 8)] \bmod 10 \\
 &= [3 + (0 \times 7)] \bmod 10 \\
 &= 3 \bmod 10 \\
 &= 3
 \end{aligned}$$

Since $\tau[3]$ is vacant, insert the key 63 in $\tau[3]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	63	24	-1	36	27	-1	-1

Step 6 Key = 81

$$\begin{aligned}
 h(81, 0) &= [81 \bmod 10 + (0 \times 81 \bmod 8)] \bmod 10 \\
 &= [1 + (0 \times 1)] \bmod 10 \\
 &= 1 \bmod 10 \\
 &= 1
 \end{aligned}$$

Since $\tau[1]$ is vacant, insert the key 81 in $\tau[1]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	81	72	63	24	-1	36	27	-1	-1

Step 7 Key = 92

$$\begin{aligned}
 h(92, 0) &= [92 \bmod 10 + (0 \times 92 \bmod 8)] \bmod 10 \\
 &= [2 + (0 \times 4)] \bmod 10 \\
 &= 2 \bmod 10 \\
 &= 2
 \end{aligned}$$

Now $\tau[2]$ is occupied, so we cannot store the key 92 in $\tau[2]$. Therefore, try again for the next location. Thus probe, $i = 1$, this time.

Key = 92

$$h(92, 1) = [92 \bmod 10 + (1 \times 92 \bmod 8)] \bmod 10$$

$$\begin{aligned}
 &= [2 + (1 \times 4)] \bmod 10 \\
 &= (2 + 4) \bmod 10 \\
 &= 6 \bmod 10 \\
 &= 6
 \end{aligned}$$

Now $\tau[6]$ is occupied, so we cannot store the key 92 in $\tau[6]$. Therefore, try again for the next location. Thus probe, $i = 2$, this time.

$$\begin{aligned}
 \text{Key} &= 92 \\
 h(92, 2) &= [92 \bmod 10 + (2 \times 92 \bmod 8)] \bmod 10 \\
 &= [2 + (2 \times 4)] \bmod 10 \\
 &= [2 + 8] \bmod 10 \\
 &= 10 \bmod 10 \\
 &= 0
 \end{aligned}$$

Since $\tau[0]$ is vacant, insert the key 92 in $\tau[0]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
92	81	72	63	24	-1	36	27	-1	-1

Step 8

$$\begin{aligned}
 \text{Key} &= 101 \\
 h(101, 0) &= [101 \bmod 10 + (0 \times 101 \bmod 8)] \bmod 10 \\
 &= [1 + (0 \times 5)] \bmod 10 \\
 &= 1 \bmod 10 \\
 &= 1
 \end{aligned}$$

Now $\tau[1]$ is occupied, so we cannot store the key 101 in $\tau[1]$. Therefore, try again for the next location. Thus probe, $i = 1$, this time.

$$\begin{aligned}
 \text{Key} &= 101 \\
 h(101, 1) &= [101 \bmod 10 + (1 \times 101 \bmod 8)] \bmod 10 \\
 &= [1 + (1 \times 5)] \bmod 10 \\
 &= [1 + 5] \bmod 10 \\
 &= 6
 \end{aligned}$$

Now $\tau[6]$ is occupied, so we cannot store the key 101 in $\tau[6]$. Therefore, try again for the next location with probe $i = 2$. Repeat the entire process until a vacant location is found. You will see that we have to probe many times to insert the key 101 in the hash table. Although double hashing is a very efficient algorithm, it always requires m to be a prime number. In our case $m=10$, which is not a prime number, hence, the degradation in performance. Had m been equal to 11, the algorithm would have worked very efficiently. Thus, we can say that the performance of the technique is sensitive to the value of m .

Rehashing

When the hash table becomes nearly full, the number of collisions increases, thereby degrading the performance of insertion and search operations. In such cases, a better option is to create a new hash table with size double of the original hash table.

All the entries in the original hash table will then have to be moved to the new hash table. This is done by taking each entry, computing its new hash value, and then inserting it in the new hash table.

Though rehashing seems to be a simple process, it is quite expensive and must therefore not be done frequently. Consider the hash table of size 5 given below. The hash function used is $h(x) = x \% 5$. Rehash the entries into to a new hash table.

0	1	2	3	4
	26	31	43	17

Note that the new hash table is of 10 locations, double the size of the original table.

0	1	2	3	4	5	6	7	8	9

Now, rehash the key values from the old hash table into the new one using hash function— $h(x) = x \% 10$.

0	1	2	3	4	5	6	7	8	9
	31		43			26	17		

PROGRAMMING EXAMPLE

1. Write a program to show searching using closed hashing.

```
#include <stdio.h>
#include <conio.h>
int ht[10], i, found = 0, key;
void insert_val();
void search_val();
void delete_val();
void display();
int main()
{
    int option;
    clrscr();
    for ( i = 0; i < 10; i++ ) //to initialize every element as '-1'
        ht[i] = -1;
    do
    {
        printf( "\n MENU \n1.Insert \n2.Search \n3.Delete \n4.Display \n5.Exit");
        printf( "\n Enter your option.");
        scanf( "%d", &option);
        switch (option)
        {
            case 1:
                insert_val();
                break;
            case 2:
                search_val();
                break;
            case 3:
                delete_val();
                break;
            case 4:
                display();
                break;
            default:
                printf( "\nInvalid choice entry!!!\n" );
                break;
        }
    }while (option!=5);
}
```



```
        getch();
        return 0;
    }
    void insert_val()
    {
        int val, f = 0;
        printf( "\nEnter the element to be inserted : " );
        scanf( "%d", &val );
        key = ( val % 10 ) - 1;
        if ( ht[key] == -1 )
        {
            ht[key] = val;
        }
        else
        {
            if ( key < 9 )
            {
                for ( i = key + 1; i < 10; i++ )
                {
                    if ( ht[i] == -1 )
                    {
                        ht[i] = val;
                        break;
                    }
                }
            }
            for ( i = 0; i < key; i++ )
            {
                if ( ht[i] == -1 )
                {
                    ht[i] = val;
                    break;
                }
            }
        }
    }
}

void display()
{
    for ( i = 0; i < 10; i++ )
        printf( "\t%d", ht[ i ] );
}

void search_val()
{
    int val, flag = 0;
    printf( "\nEnter the element to be searched :: " );
    scanf( "%d", &val );
    key = ( val % 10 ) - 1;
    if ( ht[ key ] == val )
        flag = 1;
    else
    {
        for ( i = key + 1; i < 10; i++ )
        {
            if(ht[i] == val)
            {
                flag = 1;
                key = i;
                break;
            }
        }
    }
}
```

```

    }
}
if (flag == 0)
{
    for (i = 0; i < key; i++)
    {
        if (ht[ i ] == val)
        {
            flag = 1;
            key = i;
            break;
        }
    }
}
if (flag == 1)
{
    found=1;
    printf("\n The item searched was found at position %d !", key + 1 );
}
else
{
    key = -1;
    printf( "\nThe item searched was not found in the hash table" );
}
}
void delete_val()
{
    search_val();
    if (found==1)
    {
        if ( key != -1 )
        {
            printf( "\nThe element deleted is %d ", ht[ key ] );
            ht[ key ] = -1;
        }
    }
}
}

```

Output

MENU

1. Insert
2. Search
3. Delete
4. Display
5. Exit

Enter your option: 1

Enter the element to be inserted :1

Enter your option: 4

1 -1 -1 -1 -1 -1 -1 -1 -1 -1

Enter your option: 5

15.5.2 Collision Resolution by Chaining

In chaining, each location in a hash table stores a pointer to a linked list that contains all the key values that were hashed to that location. That is, location *l* in the hash table points to the head of the linked list of all the key values that hashed to *l*. However, if no key value hashes to *l*, then location *l* in the hash table contains `NULL`. Figure 15.5 shows how the key values are mapped to a location in the hash table and stored in a linked list that corresponds to that location.

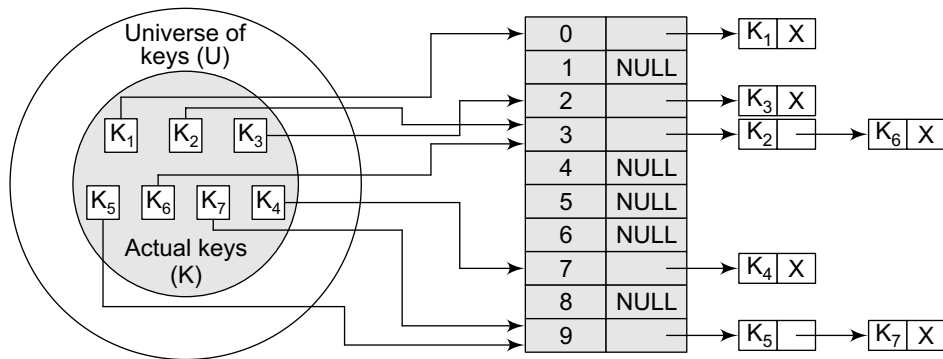


Figure 15.5 Keys being hashed to a chained hash table

Operations on a Chained Hash Table

Searching for a value in a chained hash table is as simple as scanning a linked list for an entry with the given key. Insertion operation appends the key to the end of the linked list pointed by the hashed location. Deleting a key requires searching the list and removing the element.

Chained hash tables with linked lists are widely used due to the simplicity of the algorithms to insert, delete, and search a key. The code for these algorithms is exactly the same as that for inserting, deleting, and searching a value in a single linked list that we have already studied in Chapter 6.

While the cost of inserting a key in a chained hash table is $O(1)$, the cost of deleting and searching a value is given as $O(m)$ where m is the number of elements in the list of that location. Searching and deleting takes more time because these operations scan the entries of the selected location for the desired key.

In the worst case, searching a value may take a running time of $O(n)$, where n is the number of key values stored in the chained hash table. This case arises when all the key values are inserted into the linked list of the same location (of the hash table). In this case, the hash table is ineffective.

Table 15.1 gives the code to initialize a hash table as well as the codes to insert, delete and search a value in a chained hash table.

Table 15.1 Codes to initialize, insert, delete, and search a value in a chained hash table

Structure of the node	Code to insert a value
<pre>typedef struct node_HT { int value; struct node *next; }node;</pre>	<pre>/* The element is inserted at the beginning of the linked list whose pointer to its head is stored in the location given by h(k). The run- ning time of the insert operation is O(1), as the new key value is always added as the first element of the list irrespective of the size of the linked list as well as that of the chained hash table. */ node *insert_value(node *hash_table[], int val) { node *new_node; new_node = (node *)malloc(sizeof(node)); new_node value = val; new_node next = hash_ table[h(x)]; hash_table[h(x)] = new_node; }</pre>
Code to initialize a chained hash table <pre>/* Initializes m location in the chained hash table. The operation takes a running time of O(m) */ void initializeHashTable (node *hash_ta- ble[], int m) { int i; for(i=0;i<=m;i++) hash_table[i]=NULL;</pre>	

Cont....

Cont....

Code to search a value

```

/* The element is searched in the linked
list whose pointer to its head is stored
in the location given by h(k). If search
is successful, the function returns a pointer
to the node in the linked list; otherwise
it returns NULL. The worst case running
time of the search operation is given as
order of size of the linked list. */
node *search_value(node *hash_table[],
int val)
{
    node *ptr;
    ptr = hash_table[h(x)];
    while ( (ptr!=NULL) && (ptr->value
!= val))
        ptr = ptr->next;
    if (ptr->value == val)
        return ptr;
else
    return NULL;
}

```

Code to delete a value

```

/* To delete a node from the linked list whose
head is stored at the location given by h(k)
in the hash table, we need to know the address
of the node's predecessor. We do this using a
pointer save. The running time complexity of
the delete operation is same as that of the
search operation because we need to search the
predecessor of the node so that the node can
be removed without affecting other nodes in the
list. */
void delete_value (node *hash_table[], int val)
{
    node *save, *ptr;
    save = NULL;
    ptr = hash_table[h(x)];
    while ((ptr != NULL) && (ptr->value != val))
    {
        save = ptr;
        ptr = ptr->next;
    }
    if (ptr != NULL)
    {
        save->next = ptr->next;
        free (ptr);
    }
    else
        printf("\n VALUE NOT FOUND");
}

```

Example 15.8 Insert the keys 7, 24, 18, 52, 36, 54, 11, and 23 in a chained hash table of 9 memory locations. Use $h(k) = k \bmod m$.

In this case, $m=9$. Initially, the hash table can be given as:

0	NULL
1	NULL
2	NULL
3	NULL
4	NULL
5	NULL
6	NULL
7	NULL
8	NULL

Step 1

Key = 7
 $h(k) = 7 \bmod 9$
 = 7

Create a linked list for location 7 and store the key value 7 in it as its only node.

0	NULL
1	NULL
2	NULL
3	NULL
4	NULL
5	NULL
6	NULL
7	→ [7 X]
8	NULL

Step 2

Key = 24
 $h(k) = 24 \bmod 9$
 = 6

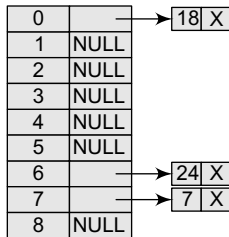
Create a linked list for location 6 and store the key value 24 in it as its only node.

0	NULL
1	NULL
2	NULL
3	NULL
4	NULL
5	NULL
6	→ [24 X]
7	→ [7 X]
8	NULL

Step 3 Key = 18

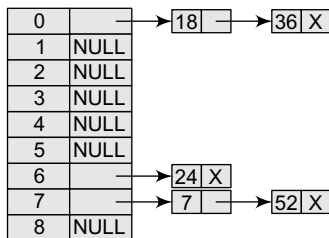
$$h(k) = 18 \bmod 9 = 0$$

Create a linked list for location 0 and store the key value 18 in it as its only node.

**Step 5:** Key = 36

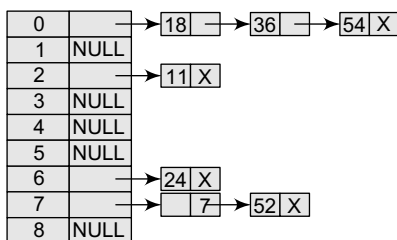
$$h(k) = 36 \bmod 9 = 0$$

Insert 36 at the end of the linked list of location 0.

**Step 7:** Key = 11

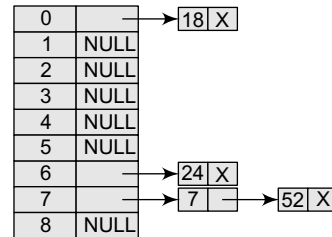
$$h(k) = 11 \bmod 9 = 2$$

Create a linked list for location 2 and store the key value 11 in it as its only node.

**Step 4** Key = 52

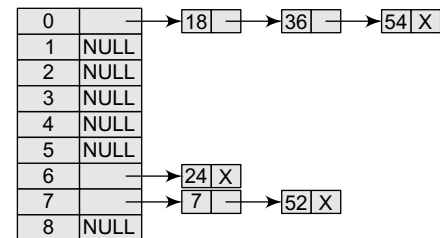
$$h(k) = 52 \bmod 9 = 7$$

Insert 52 at the end of the linked list of location 7.

**Step 6:** Key = 54

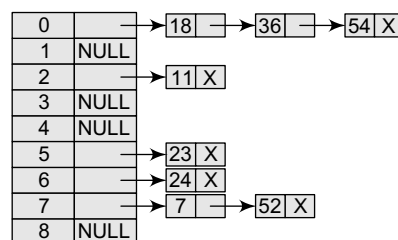
$$h(k) = 54 \bmod 9 = 0$$

Insert 54 at the end of the linked list of location 0.

**Step 8:** Key = 23

$$h(k) = 23 \bmod 9 = 5$$

Create a linked list for location 5 and store the key value 23 in it as its only node.

**Pros and Cons**

The main advantage of using a chained hash table is that it remains effective even when the number of key values to be stored is much higher than the number of locations in the hash table. However, with the increase in the number of keys to be stored, the performance of a chained hash table does degrade gradually (linearly). For example, a chained hash table with 1000 memory locations and 10,000 stored keys will give 5 to 10 times less performance as compared to a chained hash table with 10,000 locations. But a chained hash table is still 1000 times faster than a simple hash table.

The other advantage of using chaining for collision resolution is that its performance, unlike quadratic probing, does not degrade when the table is more than half full. This technique is absolutely free from clustering problems and thus provides an efficient mechanism to handle collisions.

However, chained hash tables inherit the disadvantages of linked lists. First, to store a key value, the space overhead of the next pointer in each entry can be significant. Second, traversing a linked list has poor cache performance, making the processor cache ineffective.

Bucket Hashing

In closed hashing, all the records are directly stored in the hash table. Each record with a key value k is stored in a location called its home position. The home position is calculated by applying some hash function.

In case the home position of the record with key k is already occupied by another record then the record will be stored in some other location in the hash table. This other location will be determined by the technique that is used for resolving collisions. Once the records are inserted, the same algorithm is again applied to search for a specific record.

One implementation of closed hashing groups the hash table into buckets where M slots of the hash table are divided into B buckets. Therefore, each bucket contains M/B slots. Now when a new record has to be inserted, the hash function computes the home position. If the slot is free, the record is inserted. Otherwise, the bucket's slots are sequentially searched until an open slot is found. In case, the entire bucket is full, the record is inserted into an *overflow bucket*. The overflow bucket has infinite capacity at the end of the table and is shared by all the buckets.

An efficient implementation of bucket hashing will be to use a hash function that evenly distributes the records amongst the buckets so that very few records have to be inserted in the overflow bucket.

When searching a record, first the hash function is used to determine the bucket in which the record can be present. Then the bucket is sequentially searched to find the desired record. If the record is not found and the bucket still has some empty slots, then it means that the search is complete and the desired record is not present in the hash table.

However, if the bucket is full and the record has not been found, then the overflow bucket is searched until the record is found or all the records in the overflow bucket have been checked. Obviously, searching the overflow bucket can be expensive if it has too many records.

15.6 PROS AND CONS OF HASHING

One advantage of hashing is that no extra space is required to store the index as in the case of other data structures. In addition, a hash table provides fast data access and an added advantage of rapid updates.

On the other hand, the primary drawback of using the hashing technique for inserting and retrieving data values is that it usually lacks locality and sequential retrieval by key. This makes insertion and retrieval of data values even more random.

All the more, choosing an effective hash function is more of an art than a science. It is not uncommon (in open-addressed hash tables) to create a poor hash function.

15.7 APPLICATIONS OF HASHING

Hash tables are widely used in situations where enormous amounts of data have to be accessed to quickly search and retrieve information. A few typical examples where hashing is used are given here.

Hashing is used for database indexing. Some database management systems store a separate file known as the index file. When data has to be retrieved from a file, the key information is first searched in the appropriate index file which references the exact record location of the data in the database file. This key information in the index file is often stored as a hashed value.

In many database systems, file and directory hashing is used in high-performance file systems. Such systems use two complementary techniques to improve the performance of file access. While

one of these techniques is caching which saves information in the memory, the other is hashing which makes looking up the file location in the memory much quicker than most other methods.

Hashing technique is used to implement compiler symbol tables in C++. The compiler uses a symbol table to keep a record of all the user-defined symbols in a C++ program. Hashing facilitates the compiler to quickly look up variable names and other attributes associated with symbols. Hashing is also widely used for Internet search engines.

Real World Applications of Hashing

CD Databases For CDs, it is desirable to have a world-wide CD database so that when users put their disk in the CD player, they get a full table of contents on their own computer's screen. These tables are not stored on the disks themselves, i.e., the CD does not store any information about the songs, rather this information is downloaded from the database. The critical issue to solve here is that CDs have no ID numbers stored on them, so how will the computer know which CD has been put in the player? The only information that can be used is the track length, and the fact that every CD is different.

Basically, a big number is created from the track lengths, also known as a 'signature'. This signature is used to identify a particular CD. The signature is a value obtained by hashing. For example, a number of length of 8 or 10 hexadecimal digits is made up; the number is then sent to the database, and that database looks for the closest match. The reason being that track length may not be measured exactly.

Drivers Licenses/Insurance Cards Like our CD example, even the driver's license numbers or insurance card numbers are created using hashing from data items that never change: date of birth, name, etc.

Sparse Matrix A sparse matrix is a two-dimensional array in which most of the entries contain a 0. That is, in a sparse array there are very few non-zero entries. Of course, we can store 2D array as it is, but this would lead to sheer wastage of valuable memory. So another possibility is to store the non-zero elements of the sparse matrix as elements in a 1D array. That is by using hashing, we can store a two-dimensional array in a one-dimensional array. There is a one-to-one correspondence between the elements in the sparse matrix and the elements in the array. This concept is clearly visible in Fig. 15.6.

If the size of the sparse matrix is $n \times n$, and there are N non-zero entries in it, then from the coordinates (i, j) of a matrix, we determine an index k in an array by a simple calculation. Thus, we have $k = h(i, j)$ for some function h , called a hash function.

The size of the 1D array is proportional to N . This is far better from the size of the sparse matrix that required storage proportional to $n \times n$. For example, if we have a triangular sparse matrix A , then an entry $A[i, j]$ can be mapped to an entry in the 1D array by calculating the index using the hash function $h(i, j) = i(i-1)/2 + j$.

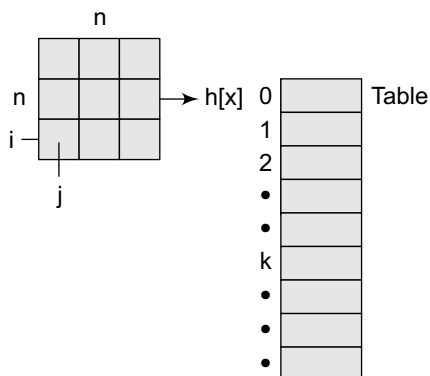


Figure 15.6 Sparse matrix

File Signatures File signatures provide a compact means of identifying files. We use a function, $h[x]$, the file signature, which is a property of the file. Although we can store files by name, signatures provide a compact identity to files.

Since a signature depends on the contents of a file, if any change is made to the file, then the signature will change. In this way, the signature of a file can be used as a quick verification to see if anyone has altered the file, or if it has lost a bit during transmission. Signatures are widely used for files that store marks of students.

Game Boards In the game board for tic-tac-toe or chess, a position in a game may be stored using a hash function.

Graphics In graphics, a central problem is the storage of objects in a scene or view. For this, we organize our data by hashing. Hashing can be used to make a grid of appropriate size, an ordinary vertical–horizontal grid. (Note that a grid is nothing but a 2D array, and there is a one-to-one correspondence when we move from a 2D array to a 1D array.)

So, we store the grid as a 1D array as we did in the case of sparse matrices. All points that fall in one cell will be stored in the same place. If a cell contains three points, then these three points will be stored in the same entry. The mapping from the grid cell to the memory location is done by using a hash function. The key advantage of this method of storage is fast execution of operations like the nearest neighbour search.

POINTS TO REMEMBER

- Hash table is a data structure in which keys are mapped to array positions by a hash function. A value stored in a hash table can be searched in $O(1)$ time using a hash function which generates an address from the key.
- The storage requirement for a hash table is $O(k)$, where k is the number of keys actually used. In a hash table, an element with key k is stored at index $h(k)$, not k . This means that a hash function h is used to calculate the index at which the element with key k will be stored. Thus, the process of mapping keys to appropriate locations (or indices) in a hash table is called hashing.
- Popular hash functions which use numeric keys are division method, multiplication method, mid square method, and folding method.
- Division method divides x by M and then uses the remainder obtained. A potential drawback of this method is that consecutive keys map to consecutive hash values.
- Multiplication method applies the hash function given as $h(x) = \lfloor m (kA \bmod 1) \rfloor$
- Mid square method works in two steps. First, it finds k^2 and then extracts the middle r digits of the result.
- Folding method works by first dividing the key value k into parts $k_1, k_2, \dots, k_n$, where each part has the same number of digits except the last part which may have lesser digits than the other parts, and then obtaining the sum of $k_1 + k_2 + \dots + k_n$. The hash value is produced by ignoring the last carry, if any.
- Collisions occur when a hash function maps two different keys to the same location. Therefore, a method used to solve the problem of collisions, also called *collision resolution technique*, is applied. The two most popular methods of resolving collisions are: (a) open addressing and (b) chaining.
- Once a collision takes place, open addressing computes new positions using a probe sequence

and the next record is stored in that position. In this technique of collision resolution, all the values are stored in the hash table. The hash table will contain two types of values—either sentinel value (for example, -1) or a data value.

- Open addressing technique can be implemented using linear probing, quadratic probing, double hashing, and rehashing.
- In linear probing, if a value is already stored at a location generated by $h(k)$, then the following hash function is used to resolve the collision:

$$h(k, i) = [h'(k) + i] \bmod m$$

Though linear probing enables good memory caching, the drawback of this algorithm is that it results in primary clustering.

- In quadratic probing, if a value is already stored at a location generated by $h(k)$, then the following hash function is used to resolve the collision:

$$h(k, i) = [h'(k) + c_1 i + c_2 i^2] \bmod m$$

Quadratic probing eliminates primary clustering and provides good memory caching. But it is still liable to secondary clustering.

- In double hashing, we use two hash functions rather than a single function. The hash function in the case of double hashing can be given as:

$$h(k, i) = [h_1(k) + i h_2(k)] \bmod m$$

The performance of double hashing is very close to the performance of the ideal scheme of uniform hashing. It minimizes repeated collisions and the effects of clustering.

- When the hash table becomes nearly full, the number of collisions increases, thereby degrading the performance of insertion and search operations. So in rehashing, all the entries in the original hash table are moved to the new hash table which is double the size of the original hash table.

- In chaining, each location in a hash table stores a pointer to a linked list that contains all the key values that were hashed to that location. While the cost of inserting a key in a chained hash table is $O(1)$, the cost

for deleting and searching a value is given as $O(m)$, where m is the number of elements in the list of that location. However, in the worst case, searching for a value may take a running time of $O(n)$.

EXERCISES

Review Questions

1. Define a hash table.
2. What do you understand by a hash function? Give the properties of a good hash function.
3. How is a hash table better than a direct access table (array)?
4. Write a short note on the different hash functions. Give suitable examples.
5. Calculate hash values of keys: 1892, 1921, 2007, 3456 using different methods of hashing.
6. What is collision? Explain the various techniques to resolve a collision. Which technique do you think is better and why?
7. Consider a hash table with size = 10. Using linear probing, insert the keys 27, 72, 63, 42, 36, 18, 29, and 101 into the table.
8. Consider a hash table with size = 10. Using quadratic probing, insert the keys 27, 72, 63, 42, 36, 18, 29, and 101 into the table. Take $c_1 = 1$ and $c_2 = 3$.
9. Consider a hash table with size = 11. Using double hashing, insert the keys 27, 72, 63, 42, 36, 18, 29, and 101 into the table. Take $h_1 = k \bmod 10$ and $h_2 = k \bmod 8$.
10. What is hashing? Give its applications. Also, discuss the pros and cons of hashing.
11. Explain chaining with examples.
12. Write short notes on:
Linear probing
Quadratic probing
Double hashing

Multiple-choice Questions

1. In a hash table, an element with key k is stored at index
(a) k (b) $\log k$ (c) $h(k)$ (d) k^2
2. In any hash function, M should be a
(a) Prime number (b) Composite number
(c) Even number (d) Odd number
3. In which of the following hash functions, do consecutive keys map to consecutive hash values?
(a) Division method
(b) Multiplication method
(c) Folding method (d) Mid-square method

4. The process of examining memory locations in a hash table is called
(a) Hashing (b) Collision
(c) Probing (d) Addressing
5. Which of the following methods is applied in the Berkeley Fast File System to allocate free blocks?
(a) Linear probing (b) Quadratic probing
(c) Double hashing (d) Rehashing
6. Which open addressing technique is free from clustering problems?
(a) Linear probing (b) Quadratic probing
(c) Double hashing (d) Rehashing

True or False

1. Hash table is based on the property of locality of reference.
2. Binary search takes $O(n \log n)$ time to execute.
3. The storage requirement for a hash table is $O(k^2)$, where k is the number of keys.
4. Hashing takes place when two or more keys map to the same memory location.
5. A good hash function completely eliminates collision.
6. M should not be too close to exact powers of 2.
7. A sentinel value indicates that the location contains valid data.
8. Linear probing is sensitive to the distribution of input values.
9. A chained hash table is faster than a simple hash table.

Fill in the Blanks

1. In a hash table, keys are mapped to array positions by a _____.
2. _____ is the process of mapping keys to appropriate locations in a hash table.
3. In open addressing, hash table stores either of two values _____ and _____.
4. When there is no free location in the hash table then _____ occurs.
5. More the number of collisions, higher is the number of _____ to find free location _____ which eliminates primary clustering but not secondary clustering.
6. _____ eliminates primary clustering but not secondary clustering.